ТПО. Лекция 2

- Жизненный цикл ПО
- Модели разработки ПО
- Жизненный цикл тестирования
- Классификация методов тестирования

ЖИЗНЕННЫЙ ЦИКЛ ПО

Это период времени, который начинается с

момента принятия решения о необходимости создания программного продукта

и заканчивается

в момент его полного изъятия из эксплуатации

Software Development Life Cycle (SDLC) – это жизненный цикл программного обеспечения из набора типовых этапов

1. **Планирование.**
2. **Анализ**
3. **Проектирование**
4. **Разработка**
5. **Тестирование и развертывание**
6. **Поддержка и сопровождение**

МОДЕЛИ РАЗРАБОТКИ ПО

Модель разработки ПО – это структура, систематизирующая различные виды проектной деятельности, их взаимодействие и последовательность в процессе разработки ПО

Выбор той или иной модели зависит от масштаба и сложности проекта, предметной области, доступных ресурсов и множества других факторов.

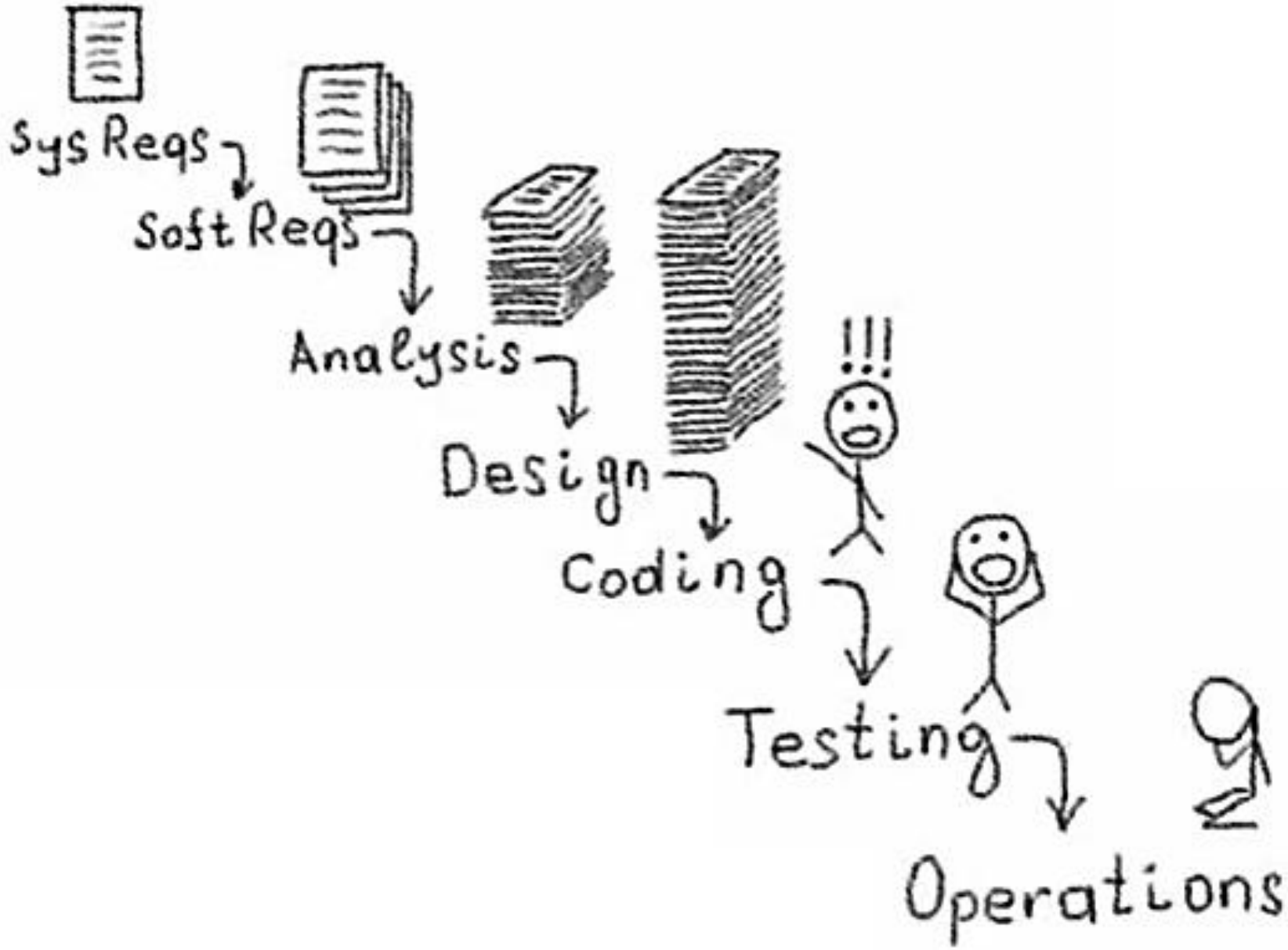
- ✓ Водопадная модель
- ✓ V-модель
- ✓ Итерационная модель
- ✓ Спиральная модель

ВОДОПАДНАЯ МОДЕЛЬ

Водопадная модель (waterfall model) сейчас представляет скорее исторический интерес, т.к. в современных проектах практически неприменима (исключением могут быть крупные проекты со стабильными требованиями).

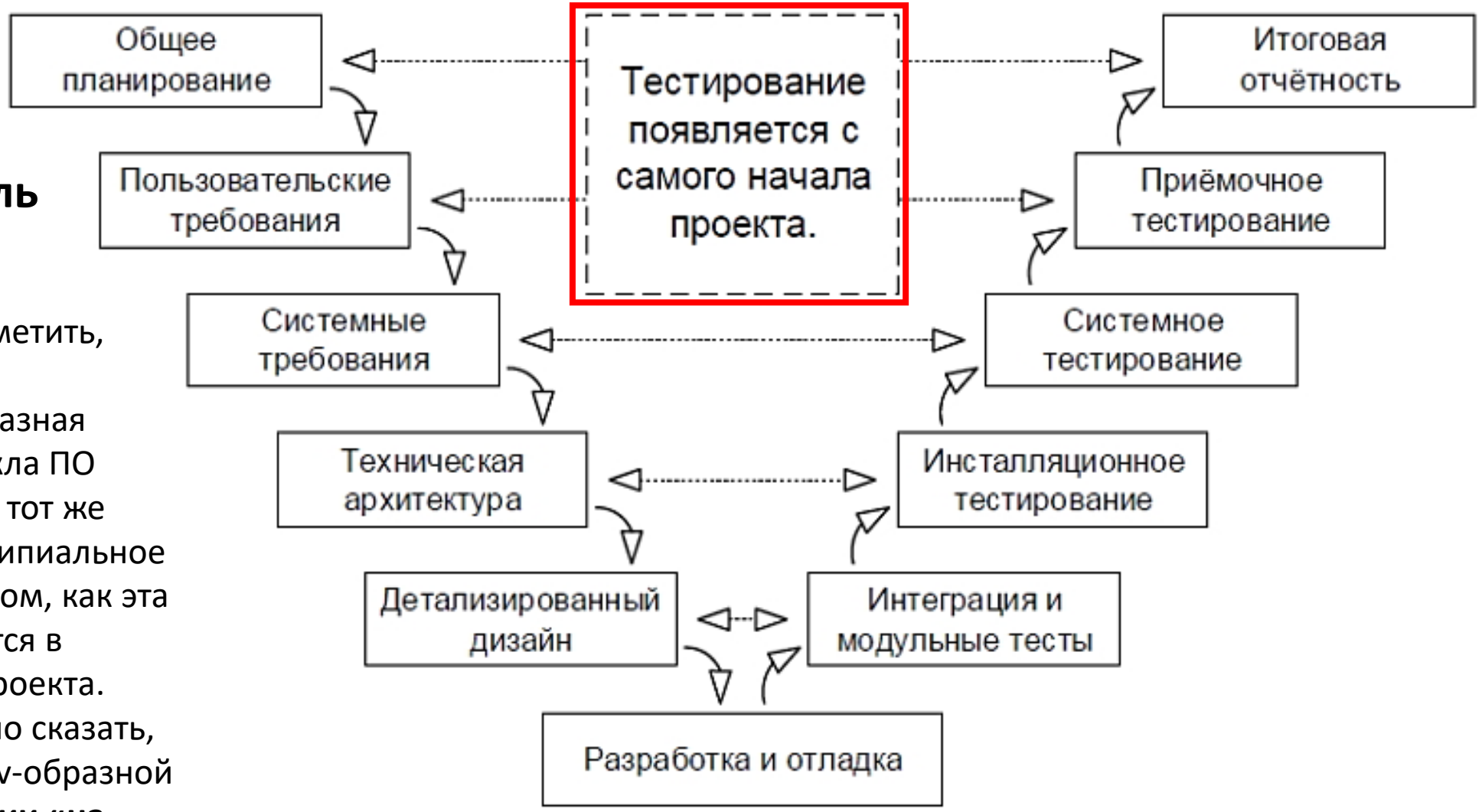
С точки зрения тестирования эта модель плоха тем, что тестирование в явном виде появляется здесь лишь с середины развития проекта, достигая своего максимума в самом конце.

!!! Забавное описание истории развития и заката водопадной модели было создано Максимом Дорофеевым в виде слайдкаста «The Rise And Fall Of Waterfall», который можно посмотреть в его ЖЖ. http://cartmendum.livejournal.com/44064.html


Sys Reqs
Soft Reqs
Analysis
Design
Coding
!!!
Testing
Operations

V-образная модель

(V-model) является логическим развитием водопадной. Можно заметить, что в общем случае как водопадная, так и v-образная модели жизненного цикла ПО могут содержать один и тот же набор стадий, но принципиальное отличие заключается в том, как эта информация используется в процессе реализации проекта. Очень упрощённо можно сказать, что при использовании v-образной модели **на каждой стадии «на спуске» нужно думать о том, что и как будет происходить на соответствующей стадии «на подъёме»**.



Тестирование появляется на ранних стадиях развития проекта. Это позволяет минимизировать риски, а также обнаружить и устранить множество потенциальных проблем.

ИТЕРАЦИОННАЯ ИНКРЕМЕНТАЛЬНАЯ МОДЕЛЬ

является фундаментальной основой современного подхода к разработке ПО. Как следует из названия модели, ей свойственна определённая двойственность:

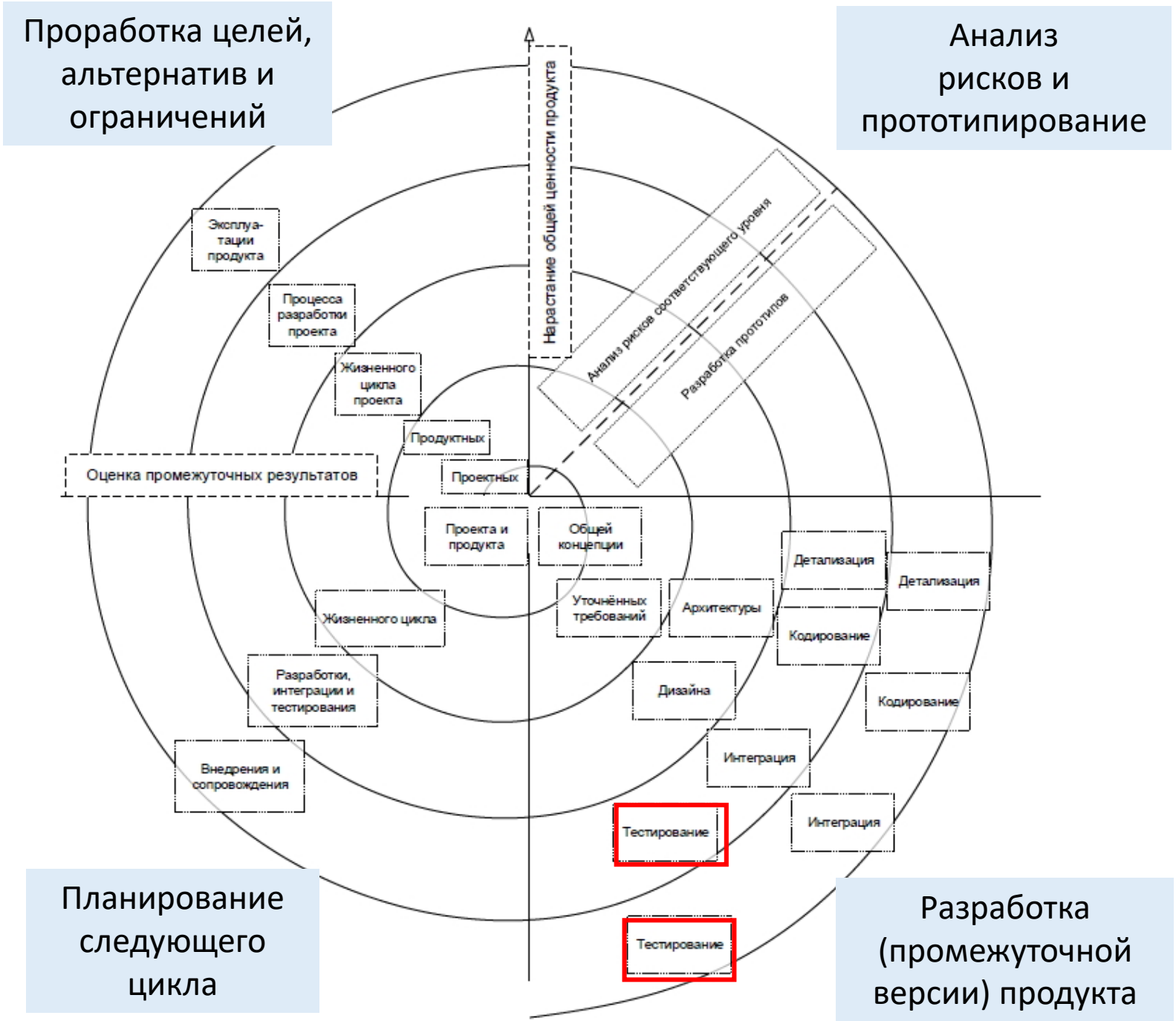
• с точки зрения жизненного цикла модель является **итерационной**, т.к. подразумевает многократное повторение одних и тех же стадий;

• с точки зрения развития продукта (приращения его полезных функций) модель является **инкрементальной**.

Ключевой особенностью данной модели является разбиение проекта на относительно небольшие промежутки (итерации), каждый из которых может включать в себя все классические стадии, присущие водопадной модели. Итогом итерации является приращение (инкремент) функциональности продукта, выраженное в промежуточном билде . Принцип многократного повторения позволяет гарантировать, что и тестирование, и демонстрация продукта конечному заказчику (с получением обратной связи) будет активно применяться с самого начала и на протяжении всего времени разработки проекта.

Спиральная модель

(spiral model) представляет собой частный случай итерационной инкрементальной модели, в котором особое внимание уделяется управлению рисками.



С точки зрения тестирования и управления качеством повышенное внимание рискам является ощутимым преимуществом при использовании спиральной модели для разработки концептуальных проектов, в которых требования могут многократно меняться по ходу выполнения проекта.

AGILE MODEL

Гибкая модель (agile model) представляет собой совокупность различных подходов к разработке ПО и базируется на т.н. «agile-манифесте»

AGILE МАНИФЕСТ:

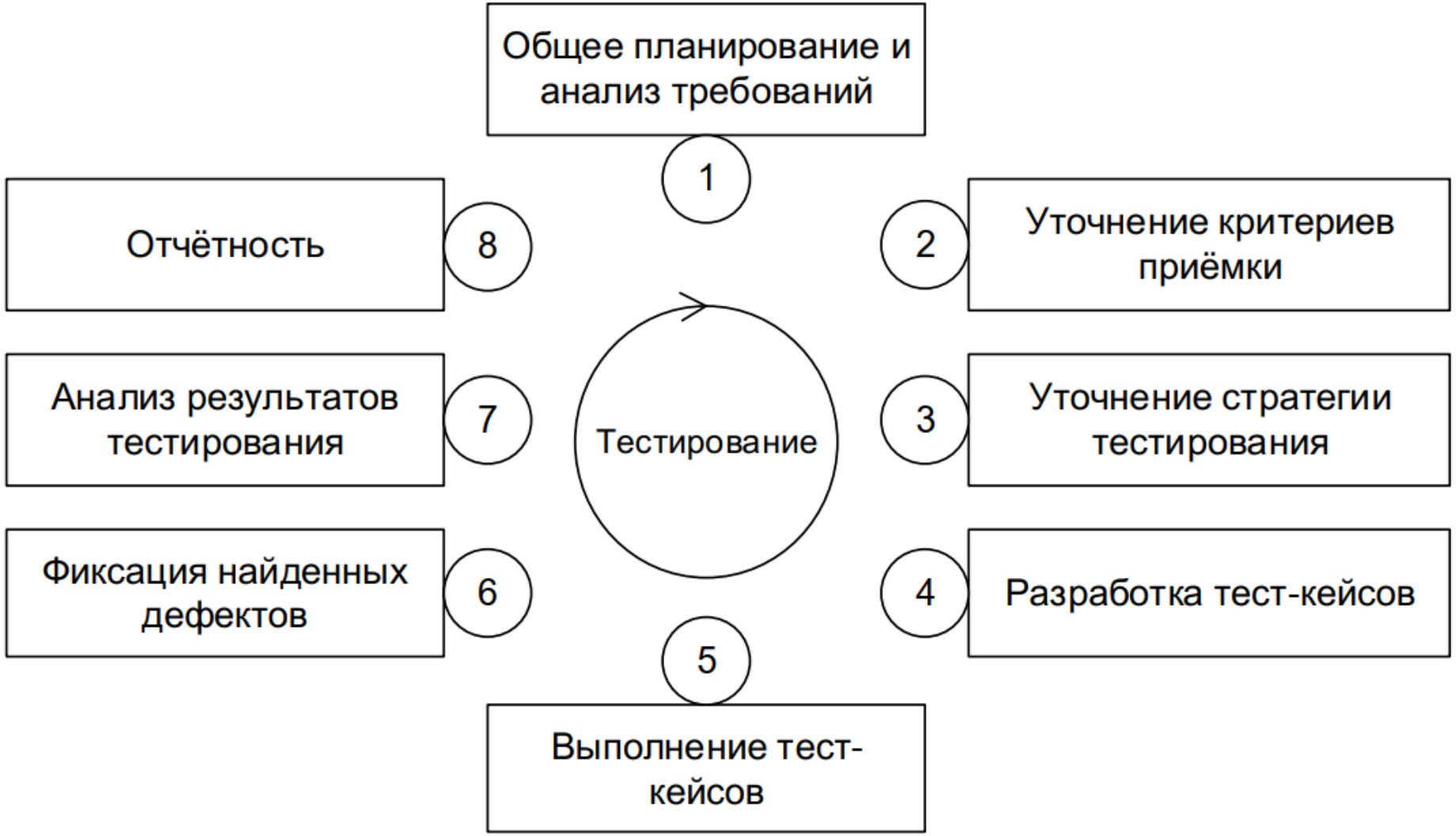
не отрицая важности того, что справа, больше ценим то, что слева.

- ✓ **Люди и взаимодействие** важнее процессов и инструментов.
- ✓ **Работающий продукт** важнее исчерпывающей документации.
- ✓ **Сотрудничество с заказчиком** важнее согласования условий контракта.
- ✓ **Готовность к изменениям** важнее следования первоначальному плану.

Сравнение моделей разработки ПО по подходу к тестированию

Модель	**Тестирование**
Водопадная	С середины проекта.
V-образная	На переходах между стадиями
Итерационная инкрементальна	• В определённые моменты итераций. • Повторное тестирование (после доработки) уже проверенного ранее.
Спиральная	
Гибкая	В определённые моменты итераций и в **любой необходимый момент**.

Жизненный цикл тестирования



Стадия 1 (общее планирование и анализ требований) Что нам предстоит тестировать; как много будет работы; какие есть сложности; всё ли необходимое у нас есть и т.п.

Стадия 2 (уточнение критериев приёмки) позволяет сформулировать или уточнить метрики и признаки возможности или необходимости начала тестирования (entry criteria), приостановки (suspension criteria) и возобновления (resumption criteria) тестирования, завершения или прекращения тестирования (exit criteria).

Стадия 3 (уточнение стратегии тестирования) планирование, но уже на локальном уровне: рассматриваются и уточняются те части стратегии тестирования (test strategy), которые актуальны для текущей итерации.

Стадия 4 (разработка тест-кейсов) посвящена разработке, пересмотру, уточнению, доработке, переработке и прочим действиям с тест-кейсами, наборами тесткейсов, тестовыми сценариями и иными артефактами, которые будут использоваться при непосредственном выполнении тестирования.

Стадия 5 (выполнение тест-кейсов) и **стадия 6 (фиксация найденных дефектов)** тесно связаны между собой и фактически выполняются параллельно: дефекты фиксируются сразу по факту их обнаружения в процессе выполнения тест-кейсов. Однако зачастую проводится явно выделенная стадия уточнения, на которой все отчёты о дефектах рассматриваются повторно с целью формирования единого понимания проблемы и уточнения таких характеристик дефекта, как важность и срочность.

Стадия 7 (анализ результатов тестирования) и **стадия 8 (отчётность)** также тесно связаны между собой и выполняются практически параллельно. Формулируемые на стадии анализа результатов выводы напрямую зависят от плана тестирования, критериев приёмки и уточнённой стратегии, полученных на стадиях 1, 2 и 3.

Полученные выводы оформляются на стадии 8 и служат основой для стадий 1, 2 и 3 следующей итерации тестирования. Таким образом, цикл замыкается.

ПРОЦЕСС ТЕСТИРОВАНИЯ

- ✓ Планирование и управление
- ✓ Анализ и проектирование
- ✓ Внедрение и реализация
- ✓ Оценка критериев выхода и создание отчетов
- ✓ Действия по завершению тестов

ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ ПО

- ✓ Анализ требований;
- ✓ Определение целей тестирования;
- ✓ Определение общего подхода к тестированию (уровни тестирования, виды тестирования, критерия входа);
- ✓ Интегрирование с разработкой ПО (требования, архитектура, дизайн, разработка, тестирование, релиз);
- ✓ Решение, какие роли нужны для выполнения тестирования, когда и как проводить тестирование и как оценивать результаты;
- ✓ Составление графика тестирования;
- ✓ Определение шаблонов для тестовой документации;
- ✓ Выбор метрик для мониторинга и контроля подготовки и проведения тестирования, исправления дефектов, проблем и рисков.

ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ ПО

- ✓ **Анализ требований;**
- ✓ Определение целей тестирования;
- ✓ Определение общего подхода к тестированию (уровни тестирования, виды тестирования, критерия входа);
- ✓ Интегрирование с разработкой ПО (требования, архитектура, дизайн, разработка, тестирование, релиз);
- ✓ Решение, какие роли нужны для выполнения тестирования, когда и как проводить тестирование и как оценивать результаты;
- ✓ Составление графика тестирования;
- ✓ Определение шаблонов для тестовой документации;
- ✓ Выбор метрик для мониторинга и контроля подготовки и проведения тестирования, исправления дефектов, проблем и рисков.

ТЕХНИКИ ТЕСТИРОВАНИЯ ТРЕБОВАНИЙ

- ✓ Взаимный просмотр
- ✓ Вопросы
- ✓ Тест-кейсы и чек-листы
- ✓ Исследование поведения системы
- ✓ Рисунки
- ✓ Прототипы

КРИТЕРИИ КАЧЕСТВА ТРЕБОВАНИЙ

- ✓ Корректность
- ✓ Недвусмысленность
- ✓ Полнота
- ✓ Непротиворечивость
- ✓ Упорядоченность по важности
- ✓ Проверяемость
- ✓ Модифицируемость
- ✓ Трассируемость
- ✓ Атомарность

Качества хороших требований

- ✓ Внимание обязательным пунктам
- ✓ Актуальность описания
- ✓ Структурированность
- ✓ Наличие указаний на необратимость действий
- ✓ Указание на ожидаемый результат
- ✓ Описание последствий отсутствия действий пользователя
- ✓ Логика и согласованность
- ✓ Последовательность изложения
- ✓ Орфография, синтаксис, пунктуация
- ✓ Описание настроек по умолчанию

Оформление вопросов к требованиям

- ✓ Вопросы пишите короткими и простыми;
- ✓ Вопрос не должен содержать «или»;
- ✓ Формулируйте вопрос так, чтобы ответ на него был максимально коротким;
- ✓ Обдумывайте ответ, который можно получить;
- ✓ Предлагая улучшения, подкрепляйте их фактами и весомыми доводами;

ПРИМЕРЫ

Требование

Приложение должно быстро запускаться.

«Плохие» вопросы

«Насколько быстро?»

(Рискуете получить ответы в стиле «очень быстро», «максимально быстро»).

«А если не получится быстро?»

(Этим вы рискуете удивить или разозлить заказчика.)

«Всегда?»

(«Да, всегда». Хм, а вы ожидали другого ответа?)

ПРИМЕРЫ

Требование

Приложение должно быстро запускаться.

«Хорошие» вопросы

«Каково максимально допустимое время запуска приложения?»

«Допускается ли фоновая загрузка отдельных компонентов приложения?»

«Что является критерием того, что приложение закончило запуск?»

ПРИМЕРЫ

Требование

Опционально должен поддерживаться экспорт документов в формат PDF.

«Плохие» вопросы

«Любых документов?»

(Ответы «да, любых» или «нет, только открытых» вам всё равно не помогут.)

«В PDF какой версии должен производиться экспорт?»

(Вопрос хороший, но он не даёт понять, что имелось в виду под «опционально».)

«Зачем?»

(«Нужно!» Именно так хочется ответить, если вопрос не раскрыт полностью.)

ПРИМЕРЫ

Требование

Опционально должен поддерживаться экспорт документов в формат PDF.

«Хорошие» вопросы

«При каких условиях будет доступен экспорт в PDF?»

«Для каких документов будет поддерживаться экспорт?»

«Какие действия пользователю необходимо выполнить, чтобы экспортировать документ в PDF?»

ПРИМЕРЫ

Требование

Если дата события не указана, она выбирается автоматически.

«Плохие» вопросы

«А если указана?»

(То она указана. Логично, не так ли?)

«А если дату невозможно выбрать автоматически?»

(Сам вопрос интересен, но без пояснения причин невозможности звучит как издёвка.)

«А если у события нет даты?»

(Автор вопроса, скорее всего, хотел уточнить, обязательно ли это поле. Но из самого требования видно, что обязательно.)

ПРИМЕРЫ

Требование

Если дата события не указана, она выбирается автоматически.

«Хороший» вопрос

«Возможно, имелось в виду, что дата **генерируется** автоматически, а не **выбирается**?

- Если «Да», то по какому алгоритму она генерируется?
- Если «Нет», то из какого набора выбирается дата и как генерируется этот набор?»

Пример оформления вопросов

Системные характеристики

- *СХ-1: Приложение является консольным.*
- *СХ-2: Для работы приложение использует интерпретатор PHP. ???*
- *СХ-3: Приложение является кроссплатформенным.*

Пользовательские требования

- *Также см. диаграмму вариантов использования.*
- *ПТ-1: Запуск и остановка приложения.*
  - *ПТ-1.1: Запуск приложения производится из консоли командой "~~PHP~~ php converter.php параметры".*
  - *ПТ-1.2: Остановка приложения производится выполнением команды Ctrl+C в окне консоли, из которого было запущено приложение.*
- *ПТ-2: Конфигурирование приложения.*
  - *ПТ-2.1: Конфигурирование приложения сводится к указанию путей в файловой системе.*
  - *ПТ-2.2: Целевой кодировкой является UTF8.*
- *ПТ-3: Просмотр журнала работы приложения.*
  - *ПТ-3.1: В процессе работы приложение должно выводить журнал своей работы в консоль и лог-файл.*
  - *ПТ-3.2: При первом запуске приложения лог-файл создаётся, а при последующих – дописывается.*

Author
1) Какая минимальная версия интерпретатора PHP поддерживается приложением?
2) Существует ли некая специфика настройки интерпретатора PHP для корректной работы приложения?

Author
Должна ли в руководстве пользователя быть описана процедура установки и настройки интерпретатора PHP?

Author
Formatted: Russian

Author
Какие ОС должны поддерживаться? В чём цель кроссплатформенности?

Author
Какие параметры передаются скрипту при запуске? Какова реакция скрипта на:
- Отсутствие параметров.
- Неверное количество параметров.
- Неверные значения каждого из параметров.

Author
Путей к чему?

Ошибки при анализе требований

Более полный список см. в книге С.Куликова «Тестирование ПО»

. . .

Критика текста или даже его автора. Помните, что ваша задача — сделать требования лучше, а не показать их недостатки (или недостатки автора). Потому комментарии вида «плохое требование», «неужели вы не понимаете, как глупо это звучит», «надо переформулировать» неуместны и недопустимы.

Категоричные заявления без обоснования.
Заявления наподобие «это невозможно», «мы не будем этого делать», «это не нужно». Даже если вы понимаете, что требование бессмысленно или невыполнимо, эту мысль стоит сформулировать в корректной форме и дополнить вопросами, позволяющими автору документа самому принять окончательное решение. Например, «это не нужно» можно переформулировать так: «Мы сомневаемся в том, что данная функция будет востребована пользователями. Какова важность этого требования? Уверены ли вы в его необходимости?»

. . .

ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ ПО

- ✓ Анализ требований;
- ✓ Определение целей тестирования;
- ✓ Определение общего подхода к тестированию (уровни тестирования, виды тестирования, критерия входа);
- ✓ Интегрирование с разработкой ПО (требования, архитектура, дизайн, разработка, тестирование, релиз);
- ✓ Решение, какие роли нужны для выполнения тестирования, когда и как проводить тестирование и как оценивать результаты;
- ✓ Составление графика тестирования;
- ✓ Определение шаблонов для тестовой документации;
- ✓ Выбор метрик для мониторинга и контроля подготовки и проведения тестирования, исправления дефектов, проблем и рисков.

Почему цели тестирования различаются?

Тестирование зависит от контекста. Согласно принципам ISTQB*, не существует универсального подхода к тестированию — оно выполняется по-разному в разных контекстах. Например, тестирование медицинского оборудования будет кардинально отличаться от тестирования мобильного приложения для развлечений.

Тип проекта	Примерные приоритеты тестирования
Safety-critical (медицина, авиация)	Надёжность, соответствие стандартам.
E-commerce	Производительность, безопасность платежей, UX
Игры	Производительность, стабильность, пользовательский опыт
Мобильные приложения	Совместимость, батарея, офлайн-режим
Финтех/банкинг	Безопасность, аудит, соответствие регуляторам

Цели тестирования не статичны — они адаптируются под контекст проекта: его домен, риски, бюджет, сроки, методологию и ожидания стейкхолдеров.

*ISTQB (International Software Testing Qualifications Board) — это ведущая международная некоммерческая организация, определяющая стандарты, методологию и терминологию в области тестирования программного обеспечения (QA).

ПЛАНИРОВАНИЕ ТЕСТИРОВАНИЯ ПО

- ✓ Анализ требований;
- ✓ Определение целей тестирования;
- ✓ Определение общего подхода к тестированию (уровни тестирования, виды тестирования, критерия входа);
- ✓ Интегрирование с разработкой ПО (требования, архитектура, дизайн, разработка, тестирование, релиз);
- ✓ Решение, какие роли нужны для выполнения тестирования, когда и как проводить тестирование и как оценивать результаты;
- ✓ Составление графика тестирования;
- ✓ Определение шаблонов для тестовой документации;
- ✓ Выбор метрик для мониторинга и контроля подготовки и проведения тестирования, исправления дефектов, проблем и рисков.

КРИТЕРИИ ВХОДА В ТЕСТИРОВАНИЕ

Критерий входа определяет, когда нужно начинать тестирование.

- ✓ Готовность и доступность тестового окружения;
- ✓ Готовность средства тестирования в окружении;
- ✓ Доступность тестируемого кода;
- ✓ Доступность тестовых данных.

КРИТЕРИИ ВЫХОДА

Критерий выхода определяет, когда нужно прекращать тестирование.

- ✓ Степень покрытие кода, функциональности или рисков тестами;
- ✓ Оценку плотности дефектов или измерение надежности;
- ✓ Стоимость;
- ✓ Остаточные риски (неисправленные дефекты или недостаток тестового покрытия какой-либо области);
- ✓ План, основанный на времени выхода ПО на рынок.

Классификация тестирования

Тестирование можно классифицировать по очень большому количеству признаков, на рисунке приведена **упрощенная классификация**

1 По запуску кода на исполнение:

- ✓ Статическое тестирование — без запуска.
- ✓ Динамическое тестирование — с запуском.

Статическое тестирование

- Тестирование документов (*требования, тест-кейсы, описания архитектуры приложения, схемы баз данных и т.д.).*
- Вычитка исходного кода (код-ревью)

Динамическое тестирование

- Модульное тестирование
- Интеграционное тестирование
- Приемочное тестирование

2 ПО СТЕПЕНИ АВТОМАТИЗАЦИИ

- ✓ Ручное тестирование;
- ✓ Автоматизированное тестирование.

Ручное тестирование (manual testing) — тестирование, в котором тест кейсы выполняются человеком вручную без использования средств автоматизации.

Автоматизированное тестирование (automated testing, test automation) — набор техник, подходов и инструментальных средств, позволяющий исклю чить человека из выполнения некоторых задач в процессе тестирования. Тест-кейсы частично или полностью выполняет специальное инструментальное средство, однако разработка тест-кейсов, подготовка данных, оценка результатов выполнения, написания отчётов об обнаруженных дефектах — всё это и многое другое по-прежнему делает человек.

3 По доступу к коду и архитектуре приложения:

- ✓ Метод белого ящика (white box testing, open box testing) — у тестировщика есть доступ к внутренней структуре и коду приложения, а также есть достаточно знаний для понимания увиденного.
- ✓ Метод чёрного ящика (black box testing, closed box testing) — у тестировщика либо нет доступа к внутренней структуре и коду приложения, либо недостаточно знаний для их понимания. Тестировщик оказывает на приложение воздействия (и проверяет реакцию) тем же способом, каким при реальной эксплуатации приложения на него воздействовали бы пользователи или другие приложения. В рамках тестирования по методу чёрного ящика основной информацией для создания тест-кейсов выступает документация и общий здравый смысл
- ✓ Метод серого ящика (gray box testing) — комбинация методов белого ящика и чёрного ящика, состоящая в том, что к части кода и архитектуры у тестировщика доступ есть, а к части — нет. Например, есть доступ к базе данных, но нет доступа к коду.

МЕТОД «БЕЛОГО ЯЩИКА»

Плюсы

- Упрощение диагностики
- Легко автоматизировать тесты
- Легко собирать данные
- Стимулирует разработчиков

Минусы

- Высокий порог входа
- Фокус только на имеющемся функционале
- Не учитывается среда выполнения
- Не учитывается непредсказуемость пользователей

МЕТОД «ЧЕРНОГО ЯЩИКА»

Плюсы

- Низкий порог входа
- Учитывается среда выполнения
- Учитывается непредсказуемость пользователей
- Раннее тестирование
- Тестирование требований
- Многократность использования

Минусы

- Повторяемость
- Вероятность неполного покрытия
- Необходимость качественной документации
- Повышенная сложность
- Трудности с планированием
- Потенциальная дороговизна

4 По уровню детализации приложения (по уровню тестирования):

- ✓ **Модульное** (компонентное) тестирование — проверяются отдельные небольшие части приложения
- ✓ **Интеграционное** тестирование — проверяется взаимодействие между несколькими частями приложения.
- ✓ **Системное** тестирование — приложение проверяется как единое целое

Скидки

Рейтинг товаров

Оплата WebMoney

Отзывы о товарах

КОМПОНЕНТНОЕ ТЕСТИРОВАНИЕ

Рейтинг товаров + Каталог товаров

Скидки + Оплата

Рейтинг товаров + Поиск товаров

Оплата WebMoney + Профайл покупателя

ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ

Каталог товаров + Поиск товаров + Оплата + Профайл покупателя + Скидки + Рейтинг товаров + Оплата WebMoney + Отзывы о товарах

СИСТЕМНОЕ ТЕСТИРОВАНИЕ

5 По (убыванию) степени важности тестируемых функций (по уровню функционального тестирования):

- ✓ **Дымовое тестирование** (smoke test) — проверка самой важной, самой ключевой функциональности, неработоспособность которой делает бессмысленной саму идею использования приложения.

- ✓ **Тестирование критического пути** (critical path test) — проверка функциональности, используемой типичными пользователями в типичной повседневной деятельности.

- ✓ **Расширенное тестирование** (extended test) — проверка всей (остальной) функциональности, заявленной в требованиях.

6 По принципам работы с приложением:

- ✓ **Позитивное тестирование** — все действия с приложением выполняются строго по инструкции без никаких недопустимых действий, некорректных данных и т.д. Можно образно сказать, что приложение исследуется в «тепличных условиях»

- ✓ **Негативное тестирование** — в работе с приложением выполняются (некорректные) операции и используются данные, потенциально приводящие к ошибкам (классика жанра — деление на ноль).

Рассмотрели упрощенную классификацию, схема и описание более подробной классификации приведена в книге «Тестирование программного обеспечения» * Святослава Куликова на стр.72

* Книга выложена на diskstation
… Тестирование ПО/Литература/Software Testing - Base Course (Svyatoslav Kulikov) - 3rd edition - RU.pdf

Классификация видов тестирования в различных источниках может немного различаться, это связано с различным уровнем проработки подобных схем их авторами.

Источник данной схемы статья
[Классификация видов тестирования / Хабр]

Регрессионное тестирование (regression testing) — тестирование, направленное на проверку того факта, что в ранее работоспособной функциональности не появились ошибки, вызванные изменениями в приложении или среде его функционирования. Фредерик Брукс в своей книге «Мифический человеко-месяц»168 писал: «Фундаментальная проблема при сопровождении программ состоит в том, что исправление одной ошибки с большой вероятностью (20–50 %) влечёт появление новой». Потому регрессионное тестирование активно используется практически в любом проекте.

Повторное тестирование (**re-testing**) — выполнение тест-кейсов, которые ранее обнаружили дефекты, с целью подтверждения устранения дефектов. Фактически этот вид тестирования сводится к действиям на финальной стадии жизненного цикла отчёта о дефекте{170}, направленным на то, чтобы перевести дефект в состояние «проверен» и «закрыт».

Санитарное тестирование (Sanity-testing) — выполняется **перед** регрессионным и **после** smoke-тестов. Целью является проверить общее состояние системы в деталях, чтобы приступить к более тщательному тестированию.

Слово ***Sanity*** переводится на русский язык как:

- вменяемость,
- здравомыслие,
- нормальная психика.

Подробнее можно прочитать в статье
[В чём разница Smoke, Sanity, Regression, Re-test и как их различать?]

Таблица 2.3.d — Виды и техники тестирования в контексте методов белого и чёрного ящиков

Вид тестирования (русскоязычное название)	Вид тестирования (англоязычное название)	Белый ящик	Чёрный ящик
Статическое тестирование[73]	Static testing	Да	Нет
Динамическое тестирование[73]	Dynamic testing	Изредка	Да
Ручное тестирование[75]	Manual testing	Мало	Да
Автоматизированное тестирование[76]	Automated testing	Да	Да
Модульное (компонентное) тестирование[77]	Unit testing, Module testing, Component testing	Да	Нет
Интеграционное тестирование[77]	Integration testing	Да	Да
Системное тестирование[78]	System testing	Мало	Да
Дымовое тестирование[79]	Smoke test, Intake test, Build verification test	Мало	Да
Тестирование критического пути[80]	Critical path test	Мало	Да